Vehicle models: use mean parameters across 3 trials

```
% Author: Jiale Jiang
% Editor: Jiale Jiang

% load the datas/workspace/structs
load("DataParser.mat","JettaData", ...
    "CamryData", "JettaMotion", "CamryMotion");
% load the latest idle rpm without replacing the saved fitted models;
SavedJetta = load("JettaData.mat", "JettaData");
SavedCamry = load("CamryData.mat", "CamryData");

display_motion_data = false;
time_step = 0.05; % s, prediction time intervals for polyval();
idle_plot_time = 15; % s, idle display time outside the course travel time;

camry_fuel_density_lb_gal = 6.17; % Referenced gasoline density, lb/US gal;

% these two lines wraps longer names into shorter names so we don't have to
% type so much;
Data = {JettaData, CamryData};
Motion = {JettaMotion, CamryMotion};

car_names = ["Jetta", "Camry"];
fuel_units = ["gal", "gal"]; % US gallons for both cars.
% fuel_fields containts the names of the two members inside the JettaData
% and CamryData cell array struct members that are related to fuel rate;
% and this will be used to loop thru each motion phase in trial for each
% car to calculate the mean fitted parameters for fuel rate vs speed;
fuel_fields = ["instantFuelRategals", "fuelrate_lbs"];

VehicleModels = cell(1,2);
% this line creates 4 structs in a cell, each containing the total
% distance and
% required target speed in each course, those values are for converting
% from miles into ft or from mph to ft/s;
Courses = {struct('total_distance',300,'target_speed',15*5280/3600), ...
    struct('total_distance',600,'target_speed',25*5280/3600), ...
    struct('total_distance',5280/4,'average_speed',15*5280/3600), ...
    struct('total_distance',5280/4)};

% use a cell array to store all the predicted results;
Predictions = cell(2,4); % Rows: Jetta/Camry; columns: Courses 1-4.
```

Average the saved motion model parameters in local phase time

reason that I chose to use the average model parameters from the fitted model is because it reduces the amount of code and plots I need to create;

```
% car = 1 is Jetta, car = 2 is Camry, so we repeat the same calculations
% for both cars without mixing their trial parameters together;
for car = 1:2

    % create a new struct for this car, then add its mean parameters below;
    model = struct('name',car_names(car),'fuel_unit',fuel_units(car));
    % Initial idle: collect one row of coefficients from each trial.
    % start with empty arrays so we can add the parameters from each trial;
    accel_coefficients = [];
    velocity_coefficients = [];
    position_coefficients = [];
    durations = NaN(3,1); % Missing phases stay NaN, not zero seconds.
    % this for loop loop thru 3 trials
    for i = 1:3
        % for idling, check if each trial contains idle first
        if isfield(Motion{car}{i}.fits, 'idle1')
            % wrap the struct name for shorter code
            fit = Motion{car}{i}.fits.idle1;
            % for each loop, add a new row to the last existing final row
            % for each trial's parameter, so the coefficients here can
            % contain up to 3 rows; each row is one trial, and each column
            % is one parameter; idle uses 1 constant parameter, while the
            % moving phases use 2 for accel, 3 for velocity, 4 for position;
            accel_coefficients(end+1,:) = fit.fitted_accel_param;
            velocity_coefficients(end+1,:) = fit.fitted_velocity_param;
            position_coefficients(end+1,:) = fit.fitted_position_param;
            durations(i) = fit.duration_sec;
        end
    end

    % ~isempty checks that we collected some parameters before averaging;
    % mean(array,1) averages down each column, so we average corresponding
    % parameters from different trials, not all the parameters together;
    if ~isempty(accel_coefficients)
        model.fits.idle1.fitted_accel_param = mean(accel_coefficients,1);
    end
    if ~isempty(velocity_coefficients)
        model.fits.idle1.fitted_velocity_param =
mean(velocity_coefficients,1);
    end
    if ~isempty(position_coefficients)
        model.fits.idle1.fitted_position_param =
mean(position_coefficients,1);
    end

    % isfinite returns true for numbers that are not NaN or Inf;
    % any checks if at least one trial has a finite duration;
    if any(isfinite(durations))
        % omitnan means ignore nan if the vectors/arrays contain any NaN;
        model.fits.idle1.duration_sec = mean(durations,'omitnan');
        % save the longest duration too; [] leaves the max dimension default;
        model.fits.idle1.max_duration_sec = max(durations,[],'omitnan');
        % nnz counts the true entries, so this counts the available trials;
        model.fits.idle1.trial_count = nnz(isfinite(durations));
```

```
end

% Acceleration: average the three trials' fitted motion parameters.
% clear the arrays so the idle parameters do not get mixed in here;
accel_coefficients = [];
velocity_coefficients = [];
position_coefficients = [];
durations = NaN(3,1);
for i = 1:3
    if isfield(Motion{car}{i}.fits, 'accel')
        fit = Motion{car}{i}.fits.accel;
        % add one trial per row, and save its acceleration duration;
        accel_coefficients(end+1,:) = fit.fitted_accel_param;
        velocity_coefficients(end+1,:) = fit.fitted_velocity_param;
        position_coefficients(end+1,:) = fit.fitted_position_param;
        durations(i) = fit.duration_sec;
    end
end
% these means give one acceleration-phase model from the three trials;
if ~isempty(accel_coefficients)
    model.fits.accel.fitted_accel_param = ...
        mean(accel_coefficients,1);
end
if ~isempty(velocity_coefficients)
    model.fits.accel.fitted_velocity_param = ...
        mean(velocity_coefficients,1);
end
if ~isempty(position_coefficients)
    model.fits.accel.fitted_position_param = ...
        mean(position_coefficients,1);
end
% save the mean duration, longest duration, and number of trials used;
if any(isfinite(durations))
    model.fits.accel.duration_sec = mean(durations,'omitnan');
    model.fits.accel.max_duration_sec = max(durations,[],'omitnan');
    model.fits.accel.trial_count = nnz(isfinite(durations));
end

% Braking: average the three trials' fitted motion coefficients.
% repeat the same process using only the braking phase parameters;
accel_coefficients = [];
velocity_coefficients = [];
position_coefficients = [];
durations = NaN(3,1);
for i = 1:3
    if isfield(Motion{car}{i}.fits, 'decel')
        fit = Motion{car}{i}.fits.decel;
        % the braking parameters are kept separate from acceleration;
        accel_coefficients(end+1,:) = fit.fitted_accel_param;
        velocity_coefficients(end+1,:) = fit.fitted_velocity_param;
        position_coefficients(end+1,:) = fit.fitted_position_param;
        durations(i) = fit.duration_sec;
    end
end
```

```
    if ~isempty(accel_coefficients)
        model.fits.decel.fitted_accel_param = ...
            mean(accel_coefficients,1);
    end
    if ~isempty(velocity_coefficients)
        model.fits.decel.fitted_velocity_param = ...
            mean(velocity_coefficients,1);
    end
    if ~isempty(position_coefficients)
        model.fits.decel.fitted_position_param = ...
            mean(position_coefficients,1);
    end
    if any(isfinite(durations))
        model.fits.decel.duration_sec = mean(durations,'omitnan');
        model.fits.decel.max_duration_sec = max(durations,[],'omitnan');
        model.fits.decel.trial_count = nnz(isfinite(durations));
    end

    % Final idle: only include trials that have this phase.
    % some trials end before we record the final idle, so isfield below
    % skips those trials instead of giving them zero-valued parameters;
    accel_coefficients = [];
    velocity_coefficients = [];
    position_coefficients = [];
    durations = NaN(3,1);
    for i = 1:3
        if isfield(Motion{car}{i}.fits, 'idle2')
            fit = Motion{car}{i}.fits.idle2;
            accel_coefficients(end+1,:) = fit.fitted_accel_param;
            velocity_coefficients(end+1,:) = fit.fitted_velocity_param;
            position_coefficients(end+1,:) = fit.fitted_position_param;
            durations(i) = fit.duration_sec;
        end
    end
    if ~isempty(accel_coefficients)
        model.fits.idle2.fitted_accel_param = mean(accel_coefficients,1);
    end
    if ~isempty(velocity_coefficients)
        model.fits.idle2.fitted_velocity_param =
mean(velocity_coefficients,1);
    end
    if ~isempty(position_coefficients)
        model.fits.idle2.fitted_position_param =
mean(position_coefficients,1);
    end
    if any(isfinite(durations))
        model.fits.idle2.duration_sec = mean(durations,'omitnan');
        model.fits.idle2.max_duration_sec = max(durations,[],'omitnan');
        model.fits.idle2.trial_count = nnz(isfinite(durations));
    end
```

Average each trial's RPM and fuel-rate coefficients

Same phase models as before: RPM/fuel rate versus speed in mph. phase holds the field name, so .(phase) selects .accel or .decel;

```
for phase = ["accel", "decel"]
    % each row is one trial, with 2 columns for slope and intercept;
    rpm_coefficients = zeros(3,2);
    fuel_coefficients = zeros(3,2);
    for i = 1:3
        % use the saved indices to get speed, rpm and fuel from the
        % same motion phase and the same trial;
        % indices are determined by the motion detection function, when
        % looping thru, this loop will first find all mean parameters
        % for idle, then for acceleration, then deceleration and
        % potentially the second idle;
        index = Motion{car}{i}.fits.(phase).indices;

        speed = Data{car}{i}.VehicleSpeed_MPH(index);

        rpm = Data{car}{i}.EngineRPM_rpm(index);

        % this contains the fuel rate for each car in each trial during
        % each motion phase;
        fuel = Data{car}{i}.(fuel_fields(car))(index);

        if car == 2
            fuel = fuel / camry_fuel_density_lb_gal; % lb/s to US gal/s
        end
        % (:) makes these column vectors so their sizes match;
        speed = speed(:);
        rpm = rpm(:);
        fuel = fuel(:);

        % select positive speed and rpm values for the rpm fit;
        valid = speed > 0 & rpm > 0;

        % polyfit() fit with linear model, and store its parameters
        rpm_coefficients(i,:) = polyfit(speed(valid),rpm(valid),1);

        % keep those selected rows, and also require fuel rate >= 0;
        valid = fuel >= 0;
        % fit the fuel rate with speed as linear model;
        fuel_coefficients(i,:) = polyfit(speed(valid),fuel(valid),1);
        % Retain the previous phase-mean fallback for negative fuel fits.
        % if the line predicts negative fuel rate at 0 or 25 mph, use
        % a constant phase mean instead; [0,mean] means zero slope;
        if any(polyval(fuel_coefficients(i,:),[0,25]) < 0)
            fuel_coefficients(i,:) = [0,mean(fuel(valid))];
        end
    end
```

```
        % average the three slopes and the three intercepts separately;
        model.rpm_param.(phase) = mean(rpm_coefficients,1);
        model.fuel_rate_param.(phase) = mean(fuel_coefficients,1);
    end
    % use the idle rpm we calculated in DataParser for each trial;
    idle_rpm = zeros(3,1);
    for i = 1:3
        if car == 1
            idle_rpm(i) = SavedJetta.JettaData{i}.trial_ave_idle_rpm;
        else
            idle_rpm(i) = SavedCamry.CamryData{i}.engine_running_idle_rpm;
        end
    end
    model.idle_rpm = mean(idle_rpm);
    % keep the highest recorded speed so we can flag predictions outside
    % the speed range we actually tested;
    model.max_measured_speed_mph = max([ ...
        max(Data{car}{1}.VehicleSpeed_MPH), ...
        max(Data{car}{2}.VehicleSpeed_MPH), ...
        max(Data{car}{3}.VehicleSpeed_MPH)]);
    % save this car's model before the next loop starts a new model struct;
    VehicleModels{car} = model;

end
JettaModel = VehicleModels{1};
CamryModel = VehicleModels{2};
```

Course timing from the mean acceleration and braking models

Reintegrate with course initial conditions; Course 1: accelerate/cruise. Course 2: accelerate/cruise/brake. Course 3: accelerate/cruise, 15 mph overall. Course 4: accelerate only.

```
for car = 1:2
    model = VehicleModels{car};
    % use this car's mean model to calculate each of the four courses;
    for course = 1:4

        distance = Courses{course}.total_distance;
        accel_param = model.fits.accel.fitted_accel_param;
        decel_param = model.fits.decel.fitted_accel_param;
        % linear acceleration uses [jerk, initial_accel], in ft/s^3 and ft/
s^2;
        jerk = accel_param(1);
        initial_accel = accel_param(2);
        % polyint integrates the polynomial; 0 sets the initial velocity
        % and position to zero for the start of this course;
        velocity_accel = polyint(accel_param,0);
        position_accel = polyint(velocity_accel,0);

        if course <= 2
            target_speed = Courses{course}.target_speed;
            % subtract the target from the constant coefficient, then roots
```

```
        % solves v(t) - target_speed = 0 for the acceleration time;
        possible_times = roots(velocity_accel - [0,0,target_speed]);

    elseif course == 3
```

Accelerating-only average check: x(t)/t = 22 ft/s

```
        average_speed = Courses{course}.average_speed;
        % this checks when the acceleration-only average reaches 15 mph;
        % the full course can also include cruising, calculated below;
        % roots[] returns all the possible roots for a polynomial
        % function, since initial speed = 0, so the actual equation
        % used to solve for the acceleration time = jerky * 1/6*
        % time^3+ a0 *1/2 * time^2 - target speed = 0;
        % in course 3, target speed = average speed = 15mph;
        check_times = roots([jerk/6,initial_accel/2,-average_speed]);
        check_times = real(check_times(abs(imag(check_times)) < ...
            1e-9 & real(check_times) > 0));
        average_check = struct('accel_time',NaN,'accel_distance',NaN, ...
            'final_speed_mph',NaN,'within_course_distance',false);
        if ~isempty(check_times)
            average_check.accel_time = min(check_times);

            average_check.accel_distance = polyval( ...
                position_accel,average_check.accel_time);

            average_check.final_speed_mph = polyval( ...
                velocity_accel,average_check.accel_time)*3600/5280;

            average_check.within_course_distance = ...
                average_check.accel_distance <= distance;
        end

        % Full course: x(ta) + v(ta)*(T-ta) = distance, T = 60 s.
        % The acceleration-only average is NOT the full-course average.
        required_time = distance/average_speed;
        % solve x(ta) + v(ta)*(required_time-ta) = distance for ta;
        % acceleration plus cruising must take 60 s for this course;
        % the cruise speed is the speed reached at ta, not exactly 15
        % mph;

        possible_times = roots([-jerk/3, ...
            jerk*required_time/2-initial_accel/2, ...
            initial_accel*required_time,-distance]);

    else
```

Course 4: keep accelerating until x(t) = 1320 ft

```
        possible_times = roots(position_accel - [0,0,0,distance]);

    end
    % roots can return negative or complex times; only keep positive
```

```
        % times with imaginary parts close to zero, allowing roundoff;
        valid_times = real(possible_times(abs(imag(possible_times)) < ...
            1e-9 & real(possible_times) > 0));
        if course == 3
            % acceleration cannot take longer than the whole course;
            valid_times = valid_times(valid_times <= required_time);
        end
        if isempty(valid_times)
            warning('%s Course %d has no feasible time.',model.name,course);
            continue;
        end
        % choose the earliest usable solution; ta is acceleration time;
        ta = min(valid_times);
        if course >= 3
            target_speed = polyval(velocity_accel,ta);
        end
        % braking starts at the reached speed, not at zero; its position
        % polynomial starts at zero here to calculate braking distance;
        velocity_decel = polyint(decel_param,target_speed);
        position_decel = polyint(velocity_decel,0);
        td = 0;
        if course == 2
            % only course 2 needs braking; solve v(t) = 0 for stopping time;
            possible_times = roots(velocity_decel);
            valid_times = real(possible_times(abs(imag(possible_times)) < ...
                1e-9 & real(possible_times) > 0));
            if isempty(valid_times)
                warning('%s Course 2 cannot stop with this
model.',model.name);
                continue;
            end
            td = min(valid_times);
        end
        % polyval evaluates the position models at the phase ending times;
        % da is acceleration distance, dd is braking distance, and dc is
        % the distance left for cruising, all in ft;
        da = polyval(position_accel,ta);
        dd = polyval(position_decel,td);
        dc = distance - da - dd;
        if abs(dc) < 1e-8
            % remove a tiny leftover distance caused by numerical roundoff;
            dc = 0;
        end
        % constant-speed travel time = remaining distance / cruising speed;
        tc = dc / target_speed;
        % phase 1 is acceleration, 2 is cruising, 3 is braking;
        phases = 1:2;
        if course == 2
            phases = 1:3;
        elseif course == 4
            phases = 1; % No cruise or braking in Course 4.
        end
```

Phase histories: acceleration, cruising, then braking if needed

Cruise RPM/fuel is an unvalidated midpoint of the two phase fits. evaluate both phase fits at the same cruise speed and average them; this is an estimate since we do not have a separate cruise fit; rpm and fuel fits use mph, so convert the predicted ft/s first;

```
        target_mph = target_speed * 3600 / 5280;
        rpm_cruise = mean([polyval(model.rpm_param.accel,target_mph), ...
            polyval(model.rpm_param.decel,target_mph)]);
        fuel_cruise = mean([polyval(model.fuel_rate_param.accel,target_mph),
...
            polyval(model.fuel_rate_param.decel,target_mph)]);
        % put the phase parameters in the same order so the loop below can
        % select acceleration, cruising, or braking using the phase number;
        acceleration_params = {accel_param, 0, decel_param};
        rpm_params = {model.rpm_param.accel, [0,rpm_cruise], ...
            model.rpm_param.decel};
        fuel_params = {model.fuel_rate_param.accel, [0,fuel_cruise], ...
            model.fuel_rate_param.decel};
        durations = [ta,tc,td];
        % these starting values connect velocity and position between phases;
        % each phase has its own local time, then starting_times places it
        % at the correct time on the complete course plot;
        starting_speeds = [0,target_speed,target_speed];
        starting_positions = [0,da,da+dc];
        starting_times = [0,ta,ta+tc];
        % start with empty vectors, then add each phase's predicted values;
        predicted = struct('time_sec',[],'position_ft',[], ...
            'speed_fts',[],'accel_fts2',[],'rpm',[],'fuel_rate',[],'phase',
[]);
        for phase = phases
            % tau is time since this phase started; linspace includes both
            % endpoints, with spacing no greater than time_step; .' turns
            % the row vector into a column vector;
            tau = linspace(0,durations(phase), ...
                max(2,ceil(durations(phase)/time_step)+1)).';
            % integrate the chosen acceleration with this phase's initial
            % velocity and position, then evaluate its velocity at tau;
            ap = acceleration_params{phase};
            vp = polyint(ap,starting_speeds(phase));
            xp = polyint(vp,starting_positions(phase));
            speed = polyval(vp,tau);
            % set the last speed exactly to the target or zero so roundoff
            % does not leave a tiny mismatch at the end of the phase;
            if phase == 1
                speed(end) = target_speed;
            elseif phase == 3
                speed(end) = 0;
            end
            % the semicolon stacks this phase's values below the previous
            % ones, so each vector contains the complete course in order;
```

```
            predicted.time_sec = [predicted.time_sec; starting_times(phase)
+tau];
            predicted.position_ft = [predicted.position_ft; polyval(xp,tau)];
            predicted.speed_fts = [predicted.speed_fts; speed];
            predicted.accel_fts2 = [predicted.accel_fts2; polyval(ap,tau)];
            % evaluate rpm and fuel from the predicted speed; max keeps rpm
            % above the selected idle floor and fuel rate at least zero;
            predicted.rpm = [predicted.rpm; max(model.idle_rpm, ...
                polyval(rpm_params{phase},speed*3600/5280))];
            predicted.fuel_rate = [predicted.fuel_rate; max(0, ...
                polyval(fuel_params{phase},speed*3600/5280))];
            % store the phase number beside each sample so we can later
            % select only acceleration, cruising, or braking when plotting;
            predicted.phase = [predicted.phase; phase*ones(size(tau))];
            predicted.accel_param{phase} = ap;
            predicted.velocity_param{phase} = vp;
            predicted.position_param{phase} = xp;
        end
        % Duplicate boundary times show phase jumps; their integration area
is 0.
        % cumtrapz adds the area under fuel rate vs time as we move along;
        % fuel rate is gal/s and time is s, so accumulated fuel is gallons;
        predicted.fuel_used =
cumtrapz(predicted.time_sec,predicted.fuel_rate);
        % leave mpg as NaN when fuel rate is zero to avoid division by zero;
        % speed / 5280 gives mi/s, then divide by gal/s to get instant mpg;
        predicted.fuel_economy = NaN(size(predicted.time_sec));
        valid = predicted.fuel_rate > 0;
        predicted.fuel_economy(valid) = predicted.speed_fts(valid) ./ ...
            (5280*predicted.fuel_rate(valid));
        predicted.economy_unit = "mi/gal";
        % save the phase times, distances and final results for the report;
        predicted.accel_time = ta;
        predicted.cruise_time = tc;
        predicted.decel_time = td;
        predicted.accel_distance = da;
        predicted.cruise_distance = dc;
        predicted.decel_distance = dd;
        predicted.total_travel_time = ta+tc+td;
        predicted.total_fuel = predicted.fuel_used(end);
        predicted.max_accel_fts2 = max(predicted.accel_fts2);
        predicted.final_speed_mph = predicted.speed_fts(end)*3600/5280;
        predicted.average_speed_mph = predicted.position_ft(end) / ...
            predicted.total_travel_time*3600/5280;

        % idle rpm is constant; keep these times separate from travel/fuel;
        predicted.idle1_time_sec = linspace(-idle_plot_time,0, ...
            max(2,ceil(idle_plot_time/time_step)+1)).';
        predicted.idle1_rpm = model.idle_rpm .* ...
            ones(size(predicted.idle1_time_sec));
        predicted.idle2_time_sec = [];
        predicted.idle2_rpm = [];
        % only course 2 stops, so only course 2 has a final idle section;
        if course == 2
```

```
            predicted.idle2_time_sec = predicted.total_travel_time + ...
                linspace(0,idle_plot_time, ...
                max(2,ceil(idle_plot_time/time_step)+1)).';
            predicted.idle2_rpm = model.idle_rpm .* ...
                ones(size(predicted.idle2_time_sec));
        end

        % flag when we accelerate longer or reach a higher speed than our
        % recorded trials; this does not check every possible model limit;
        predicted.extrapolated = ta > model.fits.accel.max_duration_sec ||
...
            target_mph > model.max_measured_speed_mph;
        if course == 3
            predicted.acceleration_average_check = average_check;
        end
        % save one struct per car and course, then print it if the flag is
on;
        Predictions{car,course} = predicted;
        if display_motion_data
            fprintf('\n%s Course %d (mean of 3 trials)\n',model.name,course);
            fprintf('Accel / cruise / brake time: %.3f / %.3f / %.3f
s\n',ta,tc,td);
            fprintf('Total: %.3f s, %.3f ft, %.6f %s fuel\n', ...
                predicted.total_travel_time,predicted.position_ft(end), ...
                predicted.total_fuel,model.fuel_unit);
            fprintf('Average / final speed: %.3f / %.3f mph\n', ...
                predicted.average_speed_mph,predicted.final_speed_mph);
            fprintf('Maximum acceleration: %.3f ft/s^2\n', ...
                predicted.max_accel_fts2);
            if course == 3
                fprintf('Acceleration-only 15 mph average: %.3f s, %.3f
ft\n', ...
                    average_check.accel_time,average_check.accel_distance);
                fprintf('Within 1/4 mile: %d; full-course average uses 60
s.\n', ...
                    average_check.within_course_distance);
            end
        end
        if predicted.extrapolated
            warning('%s Course %d exceeds the test range; not validated.',
...
                model.name,course);
        end
```

```
Warning: Jetta Course 4 exceeds the test range; not validated.

Warning: Camry Course 4 exceeds the test range; not validated.
```

```
    end
end
JettaPrediction = Predictions(1,:);
CamryPrediction = Predictions(2,:);
% save these variables so the plotting script can load them directly;
```

```
save("ModelPredictionData.mat", "Predictions", "VehicleModels", "Courses");
% Run ModelPredictionPlotting next to export the graphs.
```

Published with MATLAB® R2024b